

UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL — UEMS

Parliament Graph Architecture

Análise de Redes Parlamentares via Teoria dos Grafos

Autor: Felipe Echeverria Vilhalva

Orientador: Prof. Rubens Barbosa Filho

Período analisado: 2022–2025 (Câmara dos Deputados — Brasil)

Data do relatório: 25 de maio de 2026

1. Visão Geral do Projeto

Sistema modular para análise de redes de co-autoria parlamentar, identificando estruturas de influência através de métricas de centralidade e detecção de comunidades. Implementado em Python com arquitetura em camadas (Clean Architecture), cobertura de testes automatizados e exportação para Gephi (GEXF), CSV e SQLite.

extraction → processing → core (Graph + Algorithms) → repository → visualization

Camada	Responsabilidade	Status
extraction/	Coleta de dados via API/CSV (com cache em disco)	Completo
processing/	Limpeza, transformação e conversão para objetos de domínio	Completo
core/graph.py	Construção e operações do grafo	Completo
core/algorithms/	Métricas de centralidade e detecção de comunidades	Completo
core/algorithms/validation.py	Teste de permutação null-model (validação estatística)	Completo
models/	Entidades de domínio (Deputy, Proposition, CoauthorshipEdge)	Completo
repository/	Persistência (CSV, GEXF, SQLite)	Completo
visualization/	Gráficos e relatórios	Completo

tests/

Suite de testes automatizados

132 testes,
82%

2. O Que Foi Implementado

As 16 sugestões do orientador foram analisadas. 14 foram implementadas no código. As demais são de escrita da monografia.

- ✓ **Padronização completa para inglês** — todos os identificadores, docstrings, variáveis de ambiente e comentários convertidos para inglês.
- ✓ **PROPOSITION_TYPE_WEIGHTS movido para constants.py** — pesos experimentais centralizados com nota de que são parâmetros, não constantes teóricas justificadas (feedback do orientador).
- ✓ **DEPUTY_ID_ALIASES em constants.py** — mapeamento de IDs fantasmas da API da Câmara (ex: 130398 → 220657, André Fernandes).
- ✓ **Closeness + Eigenvector centrality integrados** — adicionados ao modelo Deputy, calculados no pipeline, exportados no CSV.
- ✓ **compute_all_centralities() orquestrador** — método único que calcula todas as métricas em sequência.
- ✓ **Bug de centralidade corrigido** — substituído `self.deputies.get(id, {})` por `_resolve_deputy()` com continue explícito quando deputado não existe.
- ✓ **Cache real em disco no ChamberExtractor** — CSVs são salvos em `data/cache/` após primeiro download; logs indicam cache hit/miss.
- ✓ **Todos os print() substituídos por logger** em `graph.py` e demais módulos.
- ✓ **Stubs implementados** — `filter_parties_by_degree()`, `filter_parties_by_betweenness()` e `export_coauthorship_metrics()` implementados; `_make_request` removido.
- ✓ **Suite de testes ampliada: 73 → 132 testes** — adicionados `test_metrics.py`, `test_community_detection.py`, `test_validation.py`; cobertura do core $\geq 93\%$.
- ✓ **README atualizado** — período 2022–2025, 132 testes, 82% cobertura, novos nomes de variáveis, campos mutáveis de Deputy documentados.
- ✓ **Validação estatística (null-model)** — `assess_community_significance()` implementado: gera N grafos aleatórios preservando sequência de graus (double-edge-swap), compara modularidade, retorna p-value.
- ✓ **Filtro max_authors** — propostas com mais de 30 co-autores deputados são excluídas das arestas. Corrige densidade de 85% → ~10%, tornando o grafo cientificamente válido para detecção de comunidades.

✓ DeprecationWarning para nomes PT antigos — variáveis de ambiente antigas (ex: LEGISLATURA_ATUAL) ainda funcionam com aviso de depreciação por uma release.

3. Resultados do Pipeline — Ano 2025



Interpretação: Q = 0,619 indica estrutura de comunidades forte (Q > 0,3 é o limiar convencional; Q > 0,6 é excelente). A rede apresenta agrupamentos claros — compatível com a hipótese de que blocos partidários estruturam a co-autoria parlamentar.

Método	Modularidade (Q)	Nº de Comunidades
Louvain	0,6193	16
Label Propagation	0,4573	11

Problema resolvido: densidade excessiva

Antes do filtro max_authors=30, uma única PEC com 226 signatários criava ~25.400 pares de arestas, resultando em densidade de 85% — grafo quase completo, inútil para análise de comunidades. Após o filtro:

Métrica	Antes do filtro	Após o filtro
Arestas	124.669	5.396
Densidade	85%	~10%
Modularidade Q	inválida	0,619

4. Cobertura de Testes





Módulo de teste	Nº de testes	O que cobre
test_aresta_coautoria.py	14	CoauthorshipEdge: criação, validação, igualdade, peso
test_deputado.py	9	Deputy: criação, validação, atualização de métricas
test_proposicao.py	11	Proposition: criação, autoria, validação, igualdade
test_graph.py	26	Grafo: estrutura, pesos, centralidade, filtros de partido, aliases
test_metrics.py	9	Degree, betweenness, closeness, eigenvector centrality
test_community_detection.py	11	Louvain, Label Propagation, Spectral, modularidade
test_validation.py	7	Null-model: tipo, p-value, Q observado, significância
test_processing.py	18	Limpeza, conversão, filtro max_authors (boundary tests)
test_repository.py	14	CSV, GEXF, SQLite: exportação e integridade
TOTAL	132	

5. O Que Ainda Falta — Código

- Integrar `assess_community_significance` no `algorithms_stage`** — o módulo de validação estatística foi implementado e testado, mas ainda não é chamado no pipeline real. Precisa ser adicionado em `src/pipeline.py` para que o p-value seja calculado e logado a cada execução. Esforço estimado: ~15 min.
- Análise temporal 2022–2025** — o TCC declara o período 2022–2025 mas o pipeline hoje executa apenas 1 ano (`PILOT_LEGISLATURE=2025`). É necessário um loop em `src/main.py` que execute o pipeline para cada ano, gerando snapshots comparáveis (GEXF, CSV e SQLite separados por ano). Esforço estimado: 2–3h (inclui tempo de download dos CSVs de cada ano).

Nota sobre análise temporal: para comparar 2022 vs 2023 vs 2024 vs 2025 no TCC, os CSVs de cada ano já estão

disponíveis no portal de dados abertos da Câmara. O código já suporta múltiplos anos — basta orquestrar o loop.

6. O Que Ainda Falta — Monografia

Abaixo estão as pendências de escrita identificadas a partir das sugestões do orientador. Nenhuma delas requer alteração de código — são seções que precisam ser redigidas.

6.1 Hipóteses de Pesquisa (Introdução)

■ Declarar explicitamente as hipóteses H1, H2, H3 na introdução. Exemplo:

H1: A rede de co-autoria apresenta estrutura de comunidades não-aleatória (Q significativamente maior que o null-model).

H2: Deputados com maior centralidade de betweenness atuam como pontes entre blocos partidários distintos.

H3: A estrutura de comunidades da rede reflete alinhamentos político-partidários observáveis.

6.2 Grafo Bipartido e Projeção (Modelagem)

■ Adicionar subseção de Modelagem definindo formalmente o grafo bipartido $B = (D \cup P, E)$, onde D é o conjunto de deputados, P o conjunto de proposições e $(d, p) \in E$ se e somente se o deputado d é coautor da proposição p. Explicar que a rede de co-autoria $G = (D, E', w)$ é a **projeção unimodal** de B sobre D, onde o peso $w(d_1, d_2)$ = número de proposições compartilhadas (ponderado pelo tipo legislativo). O código já implementa isso corretamente; falta a fundamentação escrita.

6.3 Pesos dos Tipos de Proposição (Metodologia)

⚠ **Pendência importante (sugestão explícita do orientador):** escrever subseção justificando a escolha dos pesos (PL=10, PLP=5, PEC=1, etc.) e reportar uma análise de sensibilidade mostrando como a modularidade Q varia quando os pesos mudam. Os pesos são parâmetros experimentais, não constantes teóricas — isso precisa ser declarado e fundamentado na monografia, não apenas no código.

6.4 Validação Estatística — Null-Model (Metodologia e Resultados)

■ Escrever seção de Metodologia descrevendo o teste de permutação: geração de N grafos aleatórios preservando a sequência de graus (método double-edge-swap / Configuration Model), cálculo da distribuição de Q nula e comparação com Q observado para obter p-value.

■ Após rodar o pipeline completo, escrever seção de Resultados com o p-value real obtido e a conclusão sobre a significância estatística da estrutura de comunidades detectada.

6.5 Lista de Símbolos e Notação Formal (Apêndice ou seção de Fundamentação)

- Consolidar e garantir consistência da notação ao longo do texto: $G = (V, E, w)$, D (deputados), P (proposições), Q (modularidade), C (comunidades), k_i (grau do nó i), b_i (betweenness centrality), c_i (closeness), e_i (eigenvector).

6.6 Capítulo de Resultados

- Interpretar e discutir os resultados reais do pipeline nos 4 anos (2022–2025): evolução do número de comunidades, deputados com maior centralidade por período, estabilidade dos blocos ao longo das legislaturas.

6.7 Conclusão

- Responder as hipóteses H1, H2, H3 com base nos resultados obtidos. Discutir limitações (filtro max_authors, pesos não-justificados teoricamente, análise restrita a co-autoria como proxy de alinhamento). Sugerir trabalhos futuros (algoritmo de Leiden, análise temporal mais longa, dados de votação).

7. Progresso Geral

Código / Implementação	~92%
Testes automatizados	100%
Monografia (estimado)	~40%

Resumo: a parte técnica do TCC está sólida e defensável. O código implementa corretamente todas as exigências metodológicas do orientador. O que define o prazo de entrega agora é a escrita da monografia — especialmente os capítulos de Resultados e Conclusão, que dependem dos dados da análise temporal 2022–2025 (2–3h de implementação + tempo de download).

8. Como Executar

Com Docker (recomendado)

```
docker compose up --build
```

Localmente

```
pip install -r requirements.txt
pytest src/tests/ -v --cov=src
python src/main.py
```

Variável de ambiente	Padrão	Descrição
PILOT_LEGISLATURE	2025	Ano usado no modo piloto
CURRENT_LEGISLATURE	2026	Ano da legislatura corrente
MAX_AUTHORS_PER_PROPOSAL	30	Filtro de propostas com massa de signatários
MIN_COAUTHORSHIPS	3	Mínimo de co-autorias para criar aresta
NUM_COMMUNITIES	5	Número de comunidades (Spectral)
LOG_LEVEL	INFO	Nível de log